

Claude Code 最佳实践

Best practices for Claude Code

Anthropic · 2026-10-02 (最后更新 / Last updated) · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网站导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。

从配置环境到扩展并行会话，本文介绍充分发挥 Claude Code 能力的技巧与模式。

Claude Code 是一个具备自主执行能力的编程环境。与回答问题后就等待下一条消息的聊天机器人不同，Claude Code 可以读取文件、运行命令、进行修改并自主解决问题；在此期间，你可以旁观、调整方向，也可以完全离开。

这改变了你的工作方式。你不再自己编写代码再让 Claude 审查，而是描述想要的结果，由 Claude 想清楚如何实现。Claude 会探索、规划并实施。

不过，这种自主性仍然需要一个学习过程。Claude 的工作受到某些约束，你需要理解这些约束。

本指南介绍的模式，已经在 Anthropic 内部团队，以及使用 Claude Code 处理各种代码库、语言和环境工程师中被证明有效。有关 Agent 循环的工作方式，请参阅《Claude Code 如何工作》。

大多数最佳实践都基于一个约束：Claude 的上下文窗口很快就会填满，而且随着内容增多，其表现会下降。

Claude 的上下文窗口容纳整个对话，包括每一条消息、Claude 读取的每个文件，以及每条命令的输出。然而，它很快就可能被填满。单次调试会话或代码库探索，就可能生成并消耗数万 token。

这很关键，因为随着上下文被填满，大语言模型的表现会下降。当上下文窗口接近满载时，Claude 可能开始“忘记”早先的指令，或犯更多错误。上下文窗口是最需要管理的资源。要了解一次会话实际如何逐渐填满，请观看交互式演示，了解启动时加载了什么，以及每次文件读取的开销。使用自定义状态栏持续跟踪上下文使用量，并参阅《减少 token 使用量》了解相关策略。

让 Claude 有办法验证自己的工作

给 Claude 一项能够运行的检查：测试、构建，或者可供对比的截图。这决定了你是必须守着会话，还是可以放心离开。

当工作看起来完成时，Claude 就会停下来。如果没有可以运行的检查，“看起来完成”就是唯一可用的信号，而你就成了验证循环：每个错误都要等你发现。给 Claude 一个能产出通过或失败结果的检查，这个循环便能自行闭合。Claude 完成工作、运行检查、读取结果，并不断迭代，直到检查通过。

检查可以是任何能够返回信号、并让 Claude 在对话中读取的东西：测试套件、构建退出码、代码检查器、将输出与固定测试样本进行差异比较的脚本，或者与设计稿对照的浏览器截图。Claude 的检查通过后，你再亲自运行 `/verify`，在正在运行的应用中确认这些改动。

Strategy	Before	After
Provide verification criteria	"implement a function that validates email addresses"	"write a validateEmail function. example test cases: user@example.com is true, invalid is false, user@.com is false. run the tests after implementing"
Verify UI changes visually	"make the dashboard look better"	"[paste screenshot] implement this design. take a screenshot of the result and compare it to the original. list differences and fix them"
Address root causes, not symptoms	"the build is failing"	"the build fails with this error: [paste error]. fix it and verify the build succeeds. address the root cause, don't suppress the error"

有了检查之后，再决定它要在多大程度上控制任务能否停止：

- **在单条提示词中：**像上表一样，在同一条消息中要求 Claude 运行检查并迭代。
- **贯穿整个会话：**将检查设为 `/goal` 条件。独立的评估器会在每轮之后重新检查，Claude 则持续工作，直到目标得到解决。如果 Claude 停滞不前，Claude Code 最终会停止运行，但仍保留已设置的目标；请参阅 `/goal` 评估的工作方式。
- **作为确定性关卡：**Stop hook 以脚本形式运行检查，在通过之前阻止本轮结束。Stop input 说明了连续阻止的次数上限。
- **引入第二意见：**使用验证子 Agent，或会检查自身发现的动态工作流，让一个拥有新上下文的模型尝试反驳结果，使执行任务的 Agent 不再同时负责给自己打分。

每种做法都是用前期配置换取更少的持续关注。提示词方式现在就适用于任何任务。`/goal` 和 Stop hook 方式则能让无人值守的运行在你不在场时正确完成。

让 Claude 展示证据，而不只是宣称成功：例如测试输出、它运行的命令及返回结果，或结果截图。查看证据比自己重新验证更快，也适用于你没有全程旁观的会话。

先探索，再规划，最后编写代码

将研究和规划与实施分开，避免解决了错误的问题。

让 Claude 直接开始写代码，可能产出解决错误问题的实现。使用规划模式，将探索与执行分开。

推荐的工作流分为四个阶段：

1

探索

反复按 `Shift+Tab`，直到状态栏显示 `plan mode on`，即可进入规划模式；也可以用 `claude --permission-mode plan` 启动会话。Claude 会读取文件并回答问题，但不会进行修改。

claude（规划模式）

```
read /src/auth and understand how we handle sessions and login.  
also look at how we manage environment variables for secrets.
```

2

规划

让 Claude 创建详细的实施计划。

claude（规划模式）

```
I want to add Google OAuth. What files need to change?  
What's the session flow? Create a plan.
```

按 `Ctrl+G`，在文本编辑器中打开计划，以便在 Claude 继续之前直接编辑。

3

实施

通过批准计划或按 `Shift+Tab` 退出规划模式，然后让 Claude 编写代码，并依据计划进行验证。

claude

```
implement the OAuth flow from your plan. write tests for the  
callback handler, run the test suite and fix any failures.
```

4

提交

让 Claude 使用能清楚描述改动的提交说明提交代码，并创建 PR。

claude

```
commit with a descriptive message and open a PR
```

规划模式很有用，但也会增加额外开销。

对于范围明确、改动很小的任务，例如修正拼写、添加一行日志或重命名变量，直接让 Claude 执行即可。

当你不确定采用什么方案、改动涉及多个文件，或不熟悉将要修改的代码时，规划最有价值。如果你能用一句话描述整个 diff，就跳过规划。

在提示词中提供具体上下文

指令越精确，之后需要纠正的次数就越少。

Claude 可以推断意图，但不会读心。引用具体文件、说明约束，并指出可参考的示例模式。

Strategy	Before	After
Scope the task. Specify which file, what scenario, and testing preferences.	<i>"add tests for foo.py"</i>	<i>"write a test for foo.py covering the edge case where the user is logged out. avoid mocks."</i>
Point to sources. Direct Claude to the source that can answer a question.	<i>"why does ExecutionFactory have such a weird api?"</i>	<i>"look through ExecutionFactory's git history and summarize how its api came to be"</i>
Reference existing patterns. Point Claude to patterns in your codebase.	<i>"add a calendar widget"</i>	<i>"look at how existing widgets are implemented on the home page to understand the patterns. HotDogWidget.php is a good example. follow the pattern to implement a new calendar widget that lets the user select a month and paginate forwards/backwards to pick a year. build from scratch without libraries other than the ones already used in the codebase."</i>
Describe the symptom. Provide the symptom, the likely location, and what "fixed" looks like.	<i>"fix the login bug"</i>	<i>"users report that login fails after session timeout. check the auth flow in src/auth/, especially token refresh. write a failing test that reproduces the issue, then fix it"</i>

当你正在探索，并且能够接受之后再调整方向时，模糊的提示词也有用。像 `"what would you improve in this file?"` 这样的提示词，可能会让你发现原本没有想到要问的事情。

提供丰富的内容

使用 `@` 引用文件，粘贴截图或图片，或通过管道直接输入数据。

你可以通过多种方式向 Claude 提供丰富的数据：

- 使用 `@` 引用文件，而不是描述代码位于哪里。Claude 会先读取文件，再作答。
- 直接粘贴图片。将图片复制粘贴或拖放到提示词中。
- 提供 URL，用于引用文档和 API 参考资料。用 `/permissions` 将常用域名加入允许列表。
- 通过管道输入数据：运行 `cat error.log | claude -p "explain this error"`，直接发送文件内容。
- 让 Claude 自行获取所需信息。告诉 Claude 使用 Bash 命令、MCP 工具或读取文件的方式，自行取得上下文。

配置环境

几个配置步骤，就能显著提升 Claude Code 在所有会话中的效果。有关扩展功能的完整概览，以及何时使用各项功能，请参阅《扩展 Claude Code》。

编写有效的 CLAUDE.md

运行 `/init`，根据当前项目结构生成初始 CLAUDE.md 文件，再逐步完善。

CLAUDE.md 是一个特殊文件，Claude 会在每次对话开始时读取它。将 Bash 命令、代码风格和工作流规则写在其中，为 Claude 提供仅靠代码无法推断的持久上下文。

CLAUDE.md 文件没有固定格式要求，但应保持简短，方便人类阅读。例如：

CLAUDE.md

```
# Code style
- Use ES modules (import/export) syntax, not CommonJS (require)
- Destructure imports when possible (eg. import { foo } from 'bar')

# Workflow
- Be sure to typecheck when you're done making a series of code changes
- Prefer running single tests, and not the whole test suite, for performance
```

运行 `/context`，确认 Claude 已加载该文件。每次会话都会加载 CLAUDE.md，因此只放入普遍适用的内容。对于只在部分情况下相关的领域知识或工作流，改用 Skills。Claude 会按需加载它们，避免让每次对话都膨胀。

保持简洁。对每一行都问自己：“删掉它，会导致 Claude 犯错吗？”如果不会，就删掉。臃肿的 CLAUDE.md 会让 Claude 忽略你真正的指令！

✔ Include	✖ Exclude
Bash commands Claude can't guess	Anything Claude can figure out by reading code
Code style rules that differ from defaults	Standard language conventions Claude already knows
Testing instructions and preferred test runners	Detailed API documentation (link to docs instead)
Repository etiquette (branch naming, PR conventions)	Information that changes frequently
Architectural decisions specific to your project	Long explanations or tutorials
Developer environment quirks (required env vars)	File-by-file descriptions of the codebase
Common gotchas or non-obvious behaviors	Self-evident practices like “write clean code”

如果明明已有禁止规则，Claude 仍不断做你不希望它做的事，可能是文件太长，规则被淹没了。如果 Claude 问的问题在 CLAUDE.md 中已有答案，可能是措辞存在歧义。把 CLAUDE.md 当作代码来对待：出问题检查它，定期精简，并通过观察 Claude 的行为是否真正改变来验证修改。对于已提交到仓库的 CLAUDE.md，运行 `/doctor`，Claude 就会建议删减那些能够从代码库推导出来的内容。

如果 Claude 总是跳过某一条指令，只在那一行加上“IMPORTANT”之类的强调。如果强调很多行，就没有哪一行能突出出来。将 CLAUDE.md 提交到 git，让团队也能参与维护。这个文件的价值会随时间累积。

CLAUDE.md 可以使用 `@path/to/import` 语法导入其他文件。有关导入规则和 CLAUDE.md 可以放在哪些位置，请参阅《CLAUDE.md 文件》。

配置权限

要在保留控制权的同时减少提示，可以用 `/permissions` 预先批准可信工具，并用 `/sandbox` 允许沙箱中的命令不经询问直接运行。如果想亲自批准编辑和命令，则切换到 Manual 模式。

从 Claude Code v2.1.283 开始，auto 模式是交互式终端和 VS Code 会话的内置初始权限模式：一个独立的分类器模型会替你审查大多数操作，仅阻止看起来存在风险的操作，例如扩大任务范围、操作未知基础设施，或受恶意内容驱使的操作。在更早的版本中，只有 Pro、Max 和 Team 套餐默认以 auto 模式作为内置初始权限模式。

在 Manual 模式下，Claude Code 会在执行可能修改系统的操作之前询问，例如写入文件、运行 Bash 命令和使用 MCP 工具。这很安全，但也很繁琐。批准到第十次时，你可能已经只是在机械地点确认，而不是认真审查。有两种工具可以减少 Manual 模式中的打断，而且同样适用于 auto 模式：

- **权限允许列表：**允许你知道安全的特定工具操作，例如 `npm run lint` 或 `git commit`。
- **沙箱：**启用操作系统级隔离，限制文件系统和网络访问，让 Claude 在明确的边界内更自由地工作。

进一步了解权限模式、权限规则和沙箱。

使用 CLI 工具

与外部服务交互时，告诉 Claude Code 使用 `gh`、`aws`、`gcloud` 和 `sentry-cli` 等 CLI 工具。

CLI 工具是与外部服务交互时最节省上下文的方式。如果你使用 GitHub，就安装 `gh` CLI。Claude 知道如何用它创建 issue、发起拉取请求和读取评论。没有 `gh`，Claude 仍可以使用 GitHub API，但未经认证的请求经常会触发速率限制。

Claude 也很擅长学习自己还不熟悉的 CLI 工具。可以尝试这样的提示词：`Use 'foo-cli-tool --help' to learn about foo tool, then use it to solve A, B, C.`

连接 MCP 服务器

运行 `claude mcp add`，并提供服务器名称及 URL 或命令，即可连接 Notion、Figma 或数据库等外部工具。例如：`claude mcp add --transport http notion https://mcp.notion.com/mcp`。

借助 MCP 服务器，你可以让 Claude 根据问题跟踪系统中的条目实现功能、查询数据库、分析监控数据、集成 Figma 设计，以及自动执行工作流。

设置 Hooks

对于每次都必须执行、不能有任何例外的操作，使用 Hooks。

Hooks 会在 Claude 工作流的特定节点自动运行脚本。CLAUDE.md 中的指令属于建议性要求，而 Hooks 是确定性的，可以保证操作执行。

Claude 可以帮你编写 Hooks。试试这样的提示词：“编写一个 Hook，在每次文件编辑后运行 `eslint`”，或“编写一个 Hook，阻止对 `migrations` 文件夹的写入。”若要手动配置 Hooks，直接编辑

`.claude/settings.json`，并运行 `/hooks` 浏览现有配置。

创建 Skills

在 `.claude/skills/` 中创建 `SKILL.md` 文件，为 Claude 提供领域知识和可复用工作流。

Skills 使用与你的项目、团队或领域相关的特定信息扩展 Claude 的知识。Claude 会在相关时自动应用它们，你也可以通过 `/skill-name` 直接调用。

在 `.claude/skills/` 中添加一个包含 `SKILL.md` 的目录，即可创建 Skill：

`.claude/skills/api-conventions/SKILL.md`

```
---
name: api-conventions
description: REST API design conventions for our services
---
# API Conventions
- Use kebab-case for URL paths
- Use camelCase for JSON properties
- Always include pagination for list endpoints
- Version APIs in the URL path (/v1/, /v2/)
```

Skills 也可以定义由你直接调用的可重复工作流：

`.claude/skills/fix-issue/SKILL.md`

```
---
name: fix-issue
description: Fix a GitHub issue
disable-model-invocation: true
---
Analyze and fix the GitHub issue: $ARGUMENTS.

1. Use `gh issue view` to get the issue details
2. Understand the problem described in the issue
3. Search the codebase for relevant files
4. Implement the necessary changes to fix the issue
5. Write and run tests to verify the fix
6. Ensure code passes linting and type checking
7. Create a descriptive commit message
8. Push and create a PR
```

运行 `/fix-issue 1234` 即可调用它。对于具有副作用、且希望手动触发的工作流，使用 `disable-model-invocation: true`。

创建自定义子 Agent

在 `.claude/agents/` 中定义专门的助手，让 Claude 可以将独立任务委派给它们。

子 Agent 在各自的上下文中运行，并有各自允许使用的工具集。它们适合需要读取大量文件或专注于特定领域的任务，同时不会把主对话塞满。

```
---
name: security-reviewer
description: Reviews code for security vulnerabilities
tools: Read, Grep, Glob, Bash
model: opus
---
You are a senior security engineer. Review code for:
- Injection vulnerabilities (SQL, XSS, command injection)
- Authentication and authorization flaws
- Secrets or credentials in code
- Insecure data handling

Provide specific line references and suggested fixes.
```

明确告诉 Claude 使用子 Agent: “使用一个子 Agent, 审查这段代码中的安全问题。”

安装插件

运行 `/plugin` 浏览插件市场。插件可以添加 Skills、工具和集成, 无须配置。

插件将 Skills、Hooks、子 Agent 和 MCP 服务器打包成一个可安装单元, 由社区和 Anthropic 提供。如果你使用有类型系统的编程语言, 安装一个代码智能插件, 让 Claude 能够精确导航到符号, 并在编辑后自动检测错误。

有关如何在 Skills、子 Agent、Hooks 和 MCP 之间选择, 请参阅《扩展 Claude Code》。

有效沟通

把你问其他工程师的问题拿来问 Claude; 对于规模较大的功能, 在开始实现之前, 先让 Claude 向你提问并写出规格说明。

询问代码库相关问题

向 Claude 提出你会问资深工程师的问题。

熟悉新代码库时, 可以使用 Claude Code 进行学习和探索。你可以像向其他工程师请教一样提问:

- 日志机制如何工作?
- 怎样创建一个新的 API 端点?
- `foo.rs` 第 134 行的 `async move { ... }` 是做什么的?
- `CustomerOnboardingFlowImpl` 处理了哪些边界情况?
- 为什么这段代码在第 333 行调用 `foo()`, 而不是 `bar()`?

这样使用 Claude Code, 是一种有效的上手工作流, 能够缩短熟悉项目的时间, 并减轻其他工程师的负担。不需要特殊的提示技巧, 直接提问即可。

让 Claude 向你提问

对于规模较大的功能，先让 Claude 对你进行访谈。用一个最简提示词开始，并要求 Claude 使用 `AskUserQuestion` 工具向你提问。

Claude 会询问你可能还没有考虑到的事情，包括技术实现、UI/UX、边界情况和取舍。发送提示词之前，将 `[brief description]` 替换为你的功能描述。

```
I want to build [brief description]. Interview me in detail using the AskUserQuestion tool.
```

```
Ask about technical implementation, UI/UX, edge cases, concerns, and tradeoffs. Don't ask obvious questions, dig into the hard parts I might not have considered.
```

```
Keep interviewing until we've covered everything, then write a complete spec to SPEC.md.
```

规格说明完成后，启动一个新会话来实施。新会话拥有干净的上下文，可以完全聚焦实现，而你也有一份书面规格说明可供参考。

最有用的规格说明是自包含的：明确涉及哪些文件和接口，说明哪些内容不在范围内，并以一个能证明功能有效的端到端验证步骤收尾。花时间把规格说明写准确，比花时间盯着实现过程更有回报。

管理会话

对话可以持久保存，也可以回退。充分利用这一点！

尽早、频繁地纠正方向

一旦发现 Claude 偏离方向，就立即纠正。

紧密的反馈循环能带来最好的结果。虽然 Claude 偶尔能在第一次尝试时就完美解决问题，但快速纠正它，通常能更快得到更好的方案。

- **Esc**：按 `Esc` 在操作中途停止 Claude。上下文会保留，因此你可以调整方向。
- **Esc + Esc** 或 `/rewind`：连接两次 `Esc`，或运行 `/rewind`，打开回退菜单，恢复之前的对话与代码状态，或从选定消息处开始总结。
- **"Undo that"**：让 Claude 撤销它的修改。
- **/clear**：在不相关的任务之间重置上下文。包含无关上下文的长会话会降低表现。

如果在同一会话中，你已经针对同一问题纠正 Claude 超过两次，上下文里就充斥着失败方案了。运行 `/clear`，用一个更具体、融入了已学到经验的提示词重新开始。一个使用更好提示词的干净会话，几乎总比不断累积纠正信息的长会话表现更好。

积极管理上下文

在不相关的任务之间运行 `/clear`，重置上下文。

当接近上下文上限时，Claude Code 会自动压缩对话历史，保留重要代码和决策，同时释放空间。

在长会话中，Claude 的上下文窗口可能被无关对话、文件内容和命令填满。这会降低表现，有时还会分散 Claude 的注意力。

- 在任务之间经常使用 `/clear`，彻底重置上下文窗口。
- 自动压缩触发时，Claude 会总结最重要的内容，包括代码模式、文件状态和关键决策。
- 若要更精细地控制，运行 `/compact <instructions>`，例如 `/compact Focus on the API changes`。
- 若只想压缩部分对话，使用 `Esc + Esc` 或 `/rewind`，选择一个消息检查点，然后选择 **Summarize from here**（从此处开始总结）或 **Summarize up to here**（总结到此处）。前者压缩从该位置开始的后续消息，保留之前的上下文；后者压缩较早的消息，完整保留最近的消息。参阅回退菜单的总结选项。
- 在 CLAUDE.md 中用诸如 `"When compacting, always preserve the full list of modified files and any test commands"` 的指令定制压缩行为，确保关键信息在总结之后仍被保留。
- 对于无须保留在上下文中的问题，使用 `/btw`。答案不会进入对话历史，因此你可以确认细节，而不增加上下文。

使用子 Agent 开展调查

通过 `"use subagents to investigate x"` 委派研究。子 Agent 在独立上下文中探索，让主对话保持干净，专注实施。

既然上下文是根本约束，就用子 Agent 将研究过程移出主上下文。Claude 研究代码库时会读取大量文件，这些都会消耗上下文。子 Agent 在独立的上下文窗口中运行，并汇报摘要：

```
Use subagents to investigate how our authentication system handles token refresh, and whether we have any existing OAuth utilities I should reuse.
```

你也可以在 Claude 完成实现后，使用子 Agent 进行验证。参阅《添加对抗式审查步骤》。

利用检查点回退

你发送的每条开启新一轮交互的提示词，都会创建一个检查点。你可以将对话、代码或二者一起恢复到之前任意检查点。

每次修改之前，Claude 都会自动为文件创建快照，让检查点能够恢复文件。连按两次 `Escape` 或运行 `/rewind`，即可打开回退菜单。你可以仅恢复对话、仅恢复代码、同时恢复二者，或从选定消息处开始总结。详情请参阅《检查点》。

你不必小心规划每一步，也可以让 Claude 尝试有风险的方案。如果不奏效，就回退并换一种方法。检查点会随对话一起保存，因此你可以关闭终端，之后恢复会话，仍然能够回退。

检查点只跟踪通过 Claude 文件编辑工具做出的修改。通过 Bash 命令或外部进程产生的改动不会被记录。它不能替代 git。

恢复对话

使用 `/rename` 为会话命名，并像对待分支一样管理它们：每条工作线都有自己的持久上下文。

Claude Code 会在本地保存对话，因此当一个任务需要分多次完成时，你不必重新解释上下文。运行 `claude --continue` 从上次停止的地方继续，或用 `claude --resume` 从列表中选择。给会话起一个有描述性的名称，例如 `oauth-migration`，方便之后查找。完整的恢复、分支和命名控制，请参阅《管理会话》。

自动化与规模扩展

当你能高效使用一个 Claude 之后，就可以通过并行会话、非交互模式和扇出模式成倍提升产出。

运行非交互模式

在 CI、pre-commit 钩子或脚本中使用 `claude -p "prompt"`。添加 `--output-format stream-json --verbose`，即可获得流式 JSON 输出。

通过 `claude -p "your prompt"`，你可以以非交互方式运行 Claude，不出现交互式输入提示。除非传入 `-no-session-persistence`，否则该次运行仍会创建可恢复的会话。非交互模式是将 Claude 集成到 CI 流水线、pre-commit 钩子或任何自动化工作流的方式。输出格式包括纯文本、JSON 和流式 JSON，便于程序化解析结果。

```
# One-off queries
claude -p "Explain what this project does"

# Structured output for scripts
claude -p "List all API endpoints" --output-format json

# Streaming for real-time processing
claude -p "Analyze this log file" --output-format stream-json --verbose
```

第一条命令输出纯文本。`json` 格式返回一个含有 `result` 字段的 JSON 对象。`stream-json` 格式每行输出一个 JSON 对象，从 `init` 事件开始。

运行多个 Claude 会话

并行运行多个 Claude 会话，可以加快开发、进行相互隔离的实验，或启动复杂工作流。

根据你愿意亲自承担多少协调工作，选择合适的并行方式；当会话之间需要交换发现时，再加入消息通信：

- Worktrees：在相互隔离的 git 检出目录中运行独立的 CLI 会话，避免编辑冲突。
- 跨会话消息：让你自行运行的会话相互传递发现。
- 桌面应用：以可视化方式管理多个本地会话，也可以让每个会话拥有独立的 worktree。
- 在云端使用 Claude Code：默认在 Anthropic 管理的基础设施上运行会话。
- Agent 视图：研究预览功能。运行 `claude agents`，派发能够在后台持续运行的会话，并在同一屏幕上查看它们。
- Agent 团队：实验性功能，默认关闭。通过共享任务、消息通信和团队负责人，自动协调多个会话。

除了并行处理工作，多会话也支持以质量为重点的工作流。全新的上下文能改善代码审查，因为 Claude 不会偏向自己刚写出的代码。

例如，使用“编写者 / 审查者”模式：

Session A (Writer)	Session B (Reviewer)
Implement a rate limiter for our API endpoints	
	Review the rate limiter implementation in @src/middleware/rateLimiter.ts. Look for edge cases, race conditions, and consistency with our existing middleware patterns.
Here's the review feedback: [Session B output]. Address these issues.	

测试也可以采用类似做法：让一个 Claude 编写测试，再让另一个 Claude 编写能够通过测试的代码。

跨文件扇出任务

循环遍历任务，为每个任务调用 `claude -p`。使用 `--allowedTools` 限定批量操作的权限范围。

对于大规模迁移或分析，你可以将工作分配给多个并行的 Claude 调用。运行 `/batch <instruction>`，让 Claude 将改动拆分给 5 到 30 个子 Agent；每个子 Agent 都在自己的 `worktree` 中工作。如果希望改为用自己的脚本驱动扇出，则循环调用 `claude -p`：

1

生成任务列表

让 Claude 将需要迁移的文件列表写入一个文件，以供下一步的循环读取，例如使用提示词 `list all 2,000 Python files that need migrating and save the list to files.txt`。

2

编写脚本，循环遍历列表

```
for file in $(cat files.txt); do
  claude -p "Migrate $file from Python 2 to Python 3. Return OK or FAIL." \
    --allowedTools "Edit,Bash(git commit *)"
done
```

3

先在少量文件上测试，再运行全部文件

根据前 2-3 个文件上遇到的问题完善提示词，然后再处理完整集合。`--allowedTools` 参数限制 Claude 能执行的操作，在无人值守运行时尤为重要。

你也可以将 Claude 集成到现有的数据或处理流水线中：

```
claude -p "<your prompt>" --output-format json | your_command
```

使用 auto 模式自主运行

若希望不中断地执行，同时在后台进行安全检查，使用 auto 模式。分类器模型会在命令执行前审查，阻止扩大任务范围、操作未知基础设施，以及受恶意内容驱使的操作，同时允许常规工作不经提示继续进行。

```
claude --permission-mode auto -p "fix all lint errors"
```

在带有 `-p` 参数的非交互运行中，即使分类器反复阻止操作，Claude Code 也不会停止运行。有关此时会发生什么以及相关阈值，请参阅《auto 模式何时回退》。

添加对抗式审查步骤

在认定任务完成之前，让一个子 Agent 在全新的上下文中审查 diff，并报告遗漏或不足。

Claude 无人值守工作的时间越长，在认定工作完成之前进行独立检查就越重要。运行在全新子 Agent 上下文中的审查者，只能看到 diff 和你提供的标准，看不到产生改动的推理过程，因此会独立评估结果。

如果要检查正确性，运行内置的 `/code-review` Skill。它会在一个新的子 Agent 中检查当前 diff 的缺陷，并将发现返回到会话中。如果想根据计划检查 diff，则自己编写审查提示词，明确要检查的工作、作为依据的计划，以及什么才算需要报告的问题：

```
Use a subagent to review the rate limiter diff against PLAN.md. Check that every requirement is implemented, the listed edge cases have tests, and nothing outside the task's scope changed. Report gaps, not style preferences.
```

由于审查者作为子 Agent 运行，负责实现的会话会直接收到问题，能够修复后再次审查，无须你在窗口之间复制审查结果。

一个被要求寻找缺口的审查者，通常总会报告一些问题，即使工作本身没有问题，因为你给它的任务就是找缺口。追逐每一条发现会导致过度设计：增加抽象层、防御性代码，以及针对不可能发生的情况编写测试。告诉审查者只标记影响正确性或明确需求的问题，其余都视为可选建议。

避免常见的失败模式

下面这些错误很常见。尽早识别它们可以节省时间：

- **什么都往里塞的会话。**你先开始一项任务，又问 Claude 一个不相关的问题，然后回到最初的任务。上下文因此充满无关信息。

解决办法：在不相关的任务之间使用 `/clear`。

- **反复纠正。**Claude 做错了，你纠正它；仍然错，你再纠正。失败方案污染了上下文。

解决办法：两次纠正失败后，使用 `/clear`，把已经学到的经验融入一个更好的初始提示词。

- **规定过多的 CLAUDE.md。**如果 CLAUDE.md 太长，Claude 会忽略其中一半，因为重要规则被噪声淹没了。

解决办法：果断精简。如果没有某条指令，Claude 也已经能正确完成相应操作，就删掉它，或者改成 Hook。

- **信任与验证之间的落差。**Claude 产出了看起来合理的实现，却没有处理边界情况。

解决办法：始终提供验证方式，包括测试、脚本、截图。如果无法验证，就不要交付。

- **无止境的探索。**你让 Claude“调查”某件事，却没有限定范围。Claude 读取了数百个文件，填满上下文。

解决办法：严格限定调查范围，或使用子 Agent，避免探索消耗主上下文。

培养你的直觉

本指南中的模式并非一成不变。它们是通常有效的起点，但未必适用于每一种情况。

有时你应该让上下文持续累积，因为你正深入解决一个复杂问题，历史信息很有价值。有时应该跳过规划，让 Claude 自己摸索，因为任务本身具有探索性。有时模糊的提示词恰到好处，因为你想在施加约束之前，先看看 Claude 如何理解问题。

关注哪些做法有效。当 Claude 产出出色结果时，留意你做了什么：提示词结构、提供的上下文，以及所处的模式。当 Claude 表现吃力时，追问原因。是上下文噪声太多？提示词太模糊？还是任务太大，无法一次完成？

随着时间推移，你会形成任何指南都无法完整涵盖的直觉。你会知道何时应该具体明确，何时可以开放探索；何时规划，何时探索；何时清空上下文，何时让它继续累积。

相关资源

- Claude Code 如何工作：Agent 循环、工具和上下文管理。
- 扩展 Claude Code：Skills、Hooks、MCP、子 Agent 和插件。
- 常见工作流：调试、测试、PR 等任务的分步操作方法。
- CLAUDE.md：保存项目约定和持久上下文。